



UNIVERSITETI / UNIVERSITY
"ISA BOLETINI"
MITROVICË

Big Data Processing

...

PYSPARK MACHINE LEARNING - MLLIB (Part 2)

CLASSIFICATION



Examples

Example 1: Loading spam and non-spam data

Logistic Regression is a popular method to predict a categorical response. Probably one of the most common applications of the logistic regression is the message or email spam classification. In this 3-part exercise, you'll create an email spam classifier with logistic regression using Spark MLlib. Here are the brief steps for creating a spam classifier.

Create an RDD of strings representing email.

Run MLlib's feature extraction algorithms to convert text into an RDD of vectors.

Call a classification algorithm on the RDD of vectors to return a model object to classify new points.

Evaluate the model on a test dataset using one of MLlib's evaluation functions.

In the first part of the exercise, you'll load the 'spam' and 'ham' (non-spam) files into RDDs, split the emails into individual words, and look at the first element in each of the RDD.

- Create two RDDs, one for 'spam' and one for 'non-spam (ham)'.
- Split each email in 'spam' and 'non-spam' RDDs into words.
- Print the first element in the split RDD of both 'spam' and 'non-spam'.

```
1 # Load the datasets into RDDs
2 spam_rdd = sc.___(file_path_spam)
3 non_spam_rdd = sc.___(file_path_non_spam)
4
5 # Split the email messages into words
6 spam_words = spam_rdd.___(lambda email: email.split(' '))
7 non_spam_words = non_spam_rdd.___(lambda email: ____.___(' '))
8
9 # Print the first element in the split RDD
10 print("The first element in spam_words is", spam_words.___())
11 print("The first element in non_spam_words is", ____.___())
12
```

```
The first element in spam_words is You
The first element in non_spam_words is Rofl.
```

Example 2: Feature hashing and LabelPoint

After splitting the emails into words, our raw data sets 'spam' and 'non-spam' are currently composed of 1-line messages. In order to classify these messages, we need to convert text into features.

In the second part of the exercise, you'll first create a `HashingTF()` instance to map text to vectors of 200 features. Then for each message in 'spam' and 'non-spam' files you'll split them into words, and map each word to one feature. These are the features that will be used to decide whether a message is 'spam' or 'non-spam'. Next, you'll create labels for features. For a valid message, the label will be 0 (i.e. the message is not spam) and for a 'spam' message, the label will be 1 (i.e. the message is spam). Finally, you'll combine Create a `HashingTF()` instance to map email text to vectors of 200 features.

- Each message in 'spam' and 'non-spam' datasets are split into words, and each word is mapped to one feature.
- Label the features: 1 for spam, 0 for non-spam.
- Combine both the spam and non-spam samples into a single dataset.
- e both labeled datasets.

```
1 # Create a HashingTF instance with 200 features
2 tf = HashingTF(numFeatures=200)
3
4 # Map each word to one feature
5 spam_features = tf.transform(spam_words)
6 non_spam_features = tf.transform(non_spam_words)
7
8 # Label the features: 1 for spam, 0 for non-spam
9 spam_samples = spam_features.map(lambda features: LabeledPoint(1, features))
10 non_spam_samples = non_spam_features.map(lambda features: LabeledPoint(0, features))
11
12 # Combine the two datasets
13 samples = spam_samples.concatenate(non_spam_samples)
```

Example 3: Logistic Regression model training

After creating labels and features for the data, we're ready to build a model that can learn from it (training). But before you train the model, in this final part of the exercise, you'll split the data into training and test, run Logistic Regression model on the training data, and finally check the accuracy of the model trained on training data.

- Split the combined data into training and test datasets in 80:20 ratio.
- Train the Logistic Regression model with the training dataset.
- Create a prediction label from the trained model on the test dataset.
- Combine the labels in the test dataset with the labels in the prediction dataset using zip function.
- Calculate the accuracy of the trained model using original and predicted labels, and print it.

```
1 # Split the data into training and testing
2 train_samples, test_samples = samples.____([0.8, 0.2])
3
4 # Train the model
5 model = LogisticRegressionWithLBFGS.train(____)
6
7 # Create a prediction label from the test data
8 predictions = model.____(test_samples.map(lambda x: x.features))
9
10 # Combine original labels with the predicted labels
11 labels_and_preds = test_samples.map(lambda x: x.label).zip(____)
12
13 # Check the accuracy of the model on the test data
14 accuracy = labels_and_preds.filter(lambda x: x[0] == x[____]).count() / float(test_samples.count())
15 print("Model accuracy : {:.2f}".format(____))
```

Model accuracy : 0.79

CLUSTERING



Examples

Example 1: Loading and parsing the 5000 points data

Clustering is the unsupervised learning task that involves grouping objects into clusters of high similarity. Unlike the supervised tasks, where data is labeled, clustering can be used to make sense of unlabeled data. PySpark MLlib includes the popular K-means algorithm for clustering. In this 3 part exercise, you'll find out how many clusters are there in a dataset containing 5000 rows and 2 columns. For this you'll first load the data into an RDD, parse the RDD based on the delimiter, run the KMeans model, evaluate the model and finally visualize the clusters.

In the first part, you'll load the data into RDD, parse the RDD based on the delimiter, and convert the string type of the data to an integer.

- Load the 5000_points dataset into an RDD named clusterRDD.
- Transform the clusterRDD by splitting the lines based on the tab ("\t").
- Transform the split RDD to create a list of integers for the two columns.
- Confirm that there are 5000 rows in the dataset.

```
1 # Load the dataset into an RDD
2 clusterRDD = sc.____(file_path)
3
4 # Split the RDD based on tab
5 rdd_split = clusterRDD.____(lambda x: _____.split(____))
6
7 # Transform the split RDD by creating a list of integers
8 rdd_split_int = rdd_split.____(lambda x: [int(____), int(x[1])])
9
10 # Count the number of rows in RDD
11 print("There are {} rows in the rdd_split_int dataset".format(____.____()))
```

```
There are 5000 rows in the rdd_split_int dataset
```

Example 2: K-means training

Now that the RDD is ready for training, in this 2nd part, you'll test it with k's from 13 to 16 (to save computation time) and use the elbow method to choose the correct k. The idea of the elbow method is to run K-means clustering on the dataset for different values of k, calculate Within Set Sum of Squared Error (WSSSE), and select the best k based on the sudden drop in WSSSE, i.e. where the elbow occurs. Next, you'll retrain the model with the best k and finally, get the centroids (cluster centers).

- Train the KMeans model with clusters from 13 to 16 and print the WSSSE for each cluster.
- Train the KMeans model again with the best k.
- Get the Cluster Centers (centroids) of KMeans model trained with the best k.

```
1 # Train the model with clusters from 13 to 16 and compute WSSSE
2 for clst in range(13, 17):
3     model = KMeans.____(rdd_split_int, clst, seed=1)
4     WSSSE = rdd_split_int.____(lambda point: error(point)).reduce(lambda x, y: x + y)
5     print("The cluster {} has Within Set Sum of Squared Error {}".format(clst, ____))
6
7 # Train the model again with the best k
8 model = KMeans.train(rdd_split_int, k=____, seed=1)
9
10 # Get cluster centers
11 cluster_centers = model.____
```

```
The cluster 13 has Within Set Sum of Squared Error 251787626.51713783
The cluster 14 has Within Set Sum of Squared Error 257469943.64057225
The cluster 15 has Within Set Sum of Squared Error 215235374.39950493
The cluster 16 has Within Set Sum of Squared Error 167785881.85891667
```

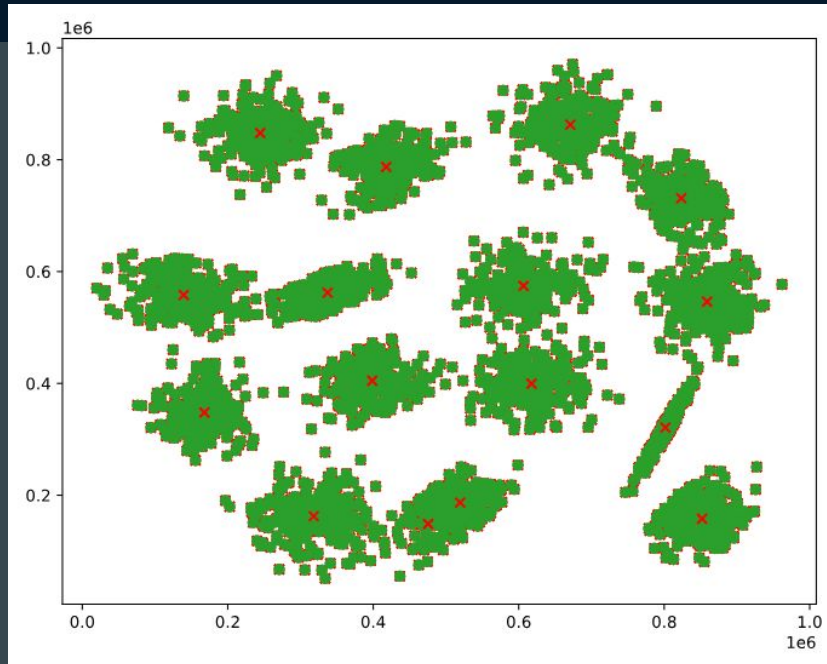
Example 3: Visualizing clusters

You just trained the k-means model with an optimum k value ($k=16$) and generated cluster centers (centroids). In this final exercise, you will visualize the clusters and the centroids by overlaying them. This will indicate how well the clustering worked (ideally, the clusters should be distinct from each other and centroids should be at the center of their respective clusters).

To achieve this, you will first convert the `rdd_split_int` RDD into a Spark DataFrame, and then into Pandas DataFrame which can be used for plotting. Similarly, you will convert `cluster_centers` into a Pandas DataFrame. Once both the DataFrames are created, you will create scatter plots using Matplotlib.

- Convert the `rdd_split_int` RDD to a Spark DataFrame, then to a pandas DataFrame.
- Create a pandas DataFrame from the `cluster_centers` list.
- Create a scatter plot from the pandas DataFrame of raw data (`rdd_split_int_df_pandas`) and overlay that with a scatter plot from the Pandas DataFrame of centroids (`cluster_centers_pandas`).

```
1 # Convert rdd_split_int RDD into Spark DataFrame and then to Pandas DataFrame
2 rdd_split_int_df_pandas = spark.____(rdd_split_int, schema=["col1", "col2"]).toPandas()
3
4 # Convert cluster_centers to a pandas DataFrame
5 cluster_centers_pandas = pd.DataFrame(____, columns=["col1", "col2"])
6
7 # Create an overlaid scatter plot of clusters and centroids
8 plt.scatter(rdd_split_int_df_pandas["col1"], rdd_split_int_df_pandas["col2"])
9 plt.scatter(____["col1"], ____["col2"], color="red", marker="x")
10 plt.show()
```





UNIVERSITETI / UNIVERSITY
"ISA BOLETINI"
MITROVICË

Stay Curious. Questions?

Thank you for your attention.
Lorent Sinani



Scan to connect on LinkedIn